

Document number: P2176r0
Date: 2020-05-27
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 (@) gmail (.) com>

A different take on inexpressible conditions

This paper proposes a solution for handling predicates in preconditions and postconditions that cannot be expressed using C++ syntax, such as "being reachable" for iterators, in a different way than what we have seen in [\[P0542r5\]](#). Rather than annotating preconditions and postconditions as inexpressible, we propose to annotate the functions used inside as inexpressible.

	Tony table
P0542r5	This proposal
<pre>bool is_null_terminated(const char *); // never defined void use_str(const char* s) [[pre: s != nullptr]] [[pre axiom: is_null_terminated(s)]]; void use_opt_str(const char* s) [[pre axiom: s == nullptr is_null_terminated(s)]]; </pre>	<pre>axiom is_null_terminated(const char *); // no definition void use_str(const char* s) [[pre: s != nullptr]] [[pre: is_null_terminated(s)]]; void use_opt_str(const char* s) [[pre: s == nullptr is_null_terminated(s)]]; </pre>

1. Motivation

There are predicates common in C++ that are defined formally in human language, but cannot be expressed as a C++ function, such as that an object of type `const char*` represents a C-style string, i.e.,

1. It is not null (this part is expressible).
2. The array under the pointer is null-terminated.

While there is no way to check this condition at runtime, expressing it using a function signature, like `is_null_terminated(str)` is still beneficial as static analyzers can make use of it by treating it as a symbol, and comparing if one function that requires an argument annotated with this symbol consumes the value produced by another function that also annotates its return value with the same symbol. Consider:

```
const char* make_name()
  [[post str: str != nullptr && is_null_terminated(str)]];
;

size_t strlen(const char* str)
  [[pre: str != nullptr && is_null_terminated(str)]];
;

size_t l = strlen(make_name());

```

Condition `str != nullptr`, which is part of the precondition of function `strlen()`, can be runtime-checked to assert (partial) program correctness. Regarding the second part, static analyzer can compare symbols: `is_null_terminated` appears as postcondition of `make_name()` and also as a precondition in `strlen()`, so assuming that `make_name()` fulfills its contract (this assumption may be verified when compiling `make_name()`), the analysis can conclude that `is_null_terminated(str)` will be satisfied in the precondition of `strlen`. Thus, function signature `is_null_terminated(const char*)` can help in determining program correctness, even if we cannot provide its body.

Note that a static analyzer can perform a similar analysis for the part `str != nullptr` of the precondition, however as such static analysis is very resource consuming, it is a reasonable strategy to perform it only for predicates of which we know that they cannot be runtime checked. So the semantics of a function signature indicated as inexpressible is, "perform symbolic static analysis for this condition, rather than trying to inject runtime defensive checks." This important use case covered in [\[P0542r5\]](#) was ignored in proposals like [\[P1429r3\]](#), which allowed specifying one of four "behaviors" for each contract declaration, but provided no way of saying, whether a given declaration should be selected for symbolic analysis or not.

2. Inexpressibility as contract statement "level"

The solution to this problem in [\[P0542r5\]](#) was to annotate the precondition or postcondition as special: designated to be treated as symbol and subject to static analysis. Because such annotation appertains to the entire predicate, if some part of a predicate can be runtime-checked and the other cannot, the predicate needs to be split across two declarations:

```
const char* make_name()
  [[post str: str != nullptr]]
  [[post axiom str: is_null_terminated(str)]];
;

size_t strlen(const char* str)
  [[pre: str != nullptr]]
  [[pre axiom: is_null_terminated(str)]];
;

```

Word `axiom` is used to indicate that this precondition needs to be verified by the static analyzer rather than runtime-checked. (At least, this was *one of* the original motivations. Other potential use cases mentioned were to annotate run-time checkable functions but with lethal side effects.)

This solution had some issues. First, it conflated the notion of "contract level" (which says about the relative runtime cost of evaluating the function body versus evaluating the predicate) with the property of requiring and not requiring the function to be ORD-used (i.e., requiring that the function should have a definition). Thus, changing the contract declaration level (from `axiom` to any other) can change a well formed program into an ill-formed one.

Second, it sometimes requires a precondition that is composed of expressible and inexpressible parts to be separated into two annotations. If the two parts are connected by a logical conjunction (operator `&&`) this is easily doable, as in the above example. However, if operator `||` is used, as in:

```
size_t consume_optional_name(const char* name)
[[pre: name == nullptr || is_null_terminated(name)]]
;
```

a separation is not possible, and the part that is easily runtime-checked must now be performed by the static analyzer, wasting its precious resources.

3. Inexpressibility as function property

The alternative that we propose is that it is the functions used in predicates that should be indicated as subject to static analysis, rather than preconditions and postconditions. This is similar to "properties" described in [\[N3351\]](#). In this proposal the examples above would be written as:

```
axiom is_null_terminated(const char* str);

const char* make_name()
[[post str: str != nullptr && is_null_terminated(str)]]
;

size_t strlen(const char* str)
[[pre: str != nullptr && is_null_terminated(str)]]
;

size_t consume_optional_name(const char* name)
[[pre: name == nullptr || is_null_terminated(name)]]
;
```

Keyword `axiom` indicates that to a certain extent `is_null_terminated` can be treated as function returning `bool`, and that its body cannot be expressed as a C++ predicate. Whenever it appears in a precondition or a postcondition, it is encouragement for the static analyzer to perform a symbolic analysis on it.

This establishes a well defined semantics of keyword `axiom`: it indicates a predicate that is meant to be consumed by the static analyzer.

An inexpressible predicate defined this way behaves like a function declaration in some aspects: it can be a template, it can be overloaded (also by a regular function), it can be looked up during overload resolution.

But it also differs from normal functions. First, it can only be used inside a contract declaration. (Later we will see a second place where it could also be useful.)

Second, because it cannot be run-time evaluated, and because contract declarations can be used to generate run-time code, special rules apply. In a run-time check, the subexpression that is marked as inexpressible is skipped: treated as if it returned `true`. This is intuitive when the entire contract predicate is inexpressible, or when the inexpressible predicate is an operand of logical conjunction:

```
void fun1(const char* str)
[[pre: str != nullptr && is_null_terminated(str)]];

void fun2(const char* str)
[[pre: str != nullptr]]
[[pre: is_null_terminated(str)]];

void test(const char* str)
{
    fun1(str);
    fun2(str);
}

// test() under certain build "mode" can render code equivalent to:

void test(const char* str)
{
    SYMBOLIC_CHECK(is_null_terminated(str));
    { if (!(str != nullptr)) contract_violation_handler(); }
    fun1(str);

    SYMBOLIC_CHECK(is_null_terminated(str));
    { if (!(str != nullptr)) contract_violation_handler(); }
    fun2(str);
}
```

However, if an inexpressible condition is an operand of logical disjunction things get complicated. The expression cannot be treated as if it returned `true` as it would incorrectly cause an expression always to succeed. In order to prevent that, we allow the inexpressible predicate only to appear on the right-hand side of an or-expression when the other operand is expressible:

```
void opt_fun1(const char* str)
[[pre: is_null_terminated(str) || str == nullptr]]; // compiler error

void opt_fun2(const char* str)
[[pre: str == nullptr || is_null_terminated(str)]];
```

```

void test(const char* str)
{
    opt_fun2(str);
}

// test() under certain build "mode" can render code equivalent to:

void test(const char* str)
{
    { if (!(str == nullptr)) SYMBOLIC_CHECK(is_null_terminated(str)); }
    opt_fun2(str);
}

```

This follows a similar approach as was taken for `requires`-clauses: logical expressions involving inexpressible predicate operands try to look like normal expressions, but they are subject to special rules. For a similar reason, you cannot negate an inexpressible predicate: requiring it would always be a logic error. You may want to know if `i` is reachable from `j` in a contract statement, but you never need to know if `i` is *unreachable* from `j`. You may want to know if a string is null-terminated, but you never want to know if it is null-not-terminated. You may want to know if a pointer points to valid memory, but never do you need to know if a pointer points to invalid memory.

You can combine expressible and inexpressible operands in one logical expression, but they have to be partitioned: first expressible, followed by inexpressible operands.

One way of looking at it is that we provide a limited form of a three-state algebra but encoded in type `bool`.

The benefit of declaring predicates as inexpressible (rather than conflating them with contract statement levels) allows the levels to be uniform: they only say when we want to evaluate them and nothing else: we do not say if the definition of a predicate may or may not be available. Even if we choose to perform "literal semantics" (like in [P1429r3](#)), the model becomes simpler: any semantic can be assigned to any contract declaration (rather than saying that for some pre-/postconditions you can only assign "ignore" semantics).

As a consequence of this, generating runtime-checks from contract annotations depends not only on "levels" or "literal semantics", but also on whether the predicates or parts thereof are marked as inexpressible.

3.1. Overloading

The fact that inexpressible predicates can be overloaded allows providing more specialized predicate overloads for which the definition can be expressed in C++. Consider predicate expressing the reachability of iterators:

```

template <input_iterator It, sentinel_for<It> S>
    axiom is_reachable(It begin, S end);

```

In case we have a special debugging-purpose iterator, it may have enough redundant information to tell if it is reachable from another iterator (like pointer to the underlying container and an index in that container). In that case we can provide an overload that is cheap to evaluate at runtime, and we will *not* mark it as inexpressible:

```

template <typename T>
bool is_reachable(vector_debug_iterator<T> begin, vector_debug_iterator<T> end)
{
    return end.is_reachable_from(begin);
}

```

Now, a function that uses the requirement `is_reachable()` does not need to know about all these overloads. It just uses the name:

```

template <input_iterator It, sentinel_for<It> S, invocable<value_type<It>> F>
void for_each(It begin, S end, F f)
    [[pre: is_reachable(begin, end)]];

```

While [P0542r5](#) also allowed to express this, it required providing an overload of an algorithm, possibly in somebody else's namespace:

```

// in P0542r5:

template <input_iterator It, sentinel_for<It> S>
bool is_reachable(It begin, S end); // never defined

template <input_iterator It, sentinel_for<It> S, invocable<value_type<It>> F>
void for_each(It begin, S end, F f)
    [[pre axiom: is_reachable(begin, end)]];

template <typename T>
bool is_reachable(vector_debug_iterator<T> begin, vector_debug_iterator<T> end)
{
    return end.is_reachable_from(begin);
}

template <typename T, invocable<value_type<vector_debug_iterator<T>>> F>
void for_each(vector_debug_iterator<T> begin, vector_debug_iterator<T> end, F f)
    [[pre: is_reachable(begin, end)]];

```

4. Syntax considerations

It is possible to provide the same functionality with less intrusive language changes:

```

bool is_null_terminated(const char* str) = axiom;

```

With this notation `axiom` can be made a context-sensitive keyword.

However, putting it at *decl-specifier* position has certain benefits. First, it makes it clear that we are defining a different entity than a function: an inexpressible predicate. It cannot be put in some places where a `bool`-returning function can appear. Second, it reflects clearer that the return type is `bool` type-system wise, but the value computation in logical expressions is different. It is only this special type that an inexpressible predicate can have, so there is no point in adding the extra flexibility for the user to specify any return type only to later inform him/her that it will not compile.

This also clearly indicates that we are sort-of implementing a three-state algebra in type `bool`.

Finally, by clearly stating that an inexpressible predicate is a distinct entity, we allow room for potential further extensions. While we do not propose these extensions now, this section indicates the future possible evolution directions.

4.1 Partial implementations

For instance, when in some special cases the value of a generally inexpressible predicate can be determined at run-time, we can allow the programmer to specify this. Consider the specialization of `is_reachable()` for contiguous iterators. When `std::less<>{}(p, q) == false` we know for sure that `p` is not reachable from `q`. In case `std::less<>{}(p, q) == true` we do not know the answer. But this partial information is better than no information, so we could express it as:

```
template <std::contiguous_iterator IT>
axiom is_reachable(IT begin, IT end)
    = !std::less<>{}(to_address(end), to_address(begin)) && std::indeterminate();
```

where `std::indeterminate()` is defined as:

```
axiom indeterminate();
```

This is the second context where we would allow the invocation of an inexpressible predicate. Owing to short-circuiting `is_reachable` defined like this could for some inputs provide a runtime result `false`. Similarly, we can define a value constraint on a C-style string:

```
axiom is_c_str(const char * s)
    = (s != nullptr) && std::indeterminate();
```

A logical disjunction in the definition of an inexpressible predicate is also possible and could return `true` for some inputs:

```
axiom is_optional_c_str(const char * s)
    = (s == nullptr) || std::indeterminate();
```

The same special rules apply in this context: inexpressible predicates can be mixed with normal predicates, but normal predicates must come first. Similarly, you cannot negate an inexpressible predicate.

4.3 Demonstration of usage

Another possible extension is to use the body of an inexpressible predicate `P` to describe a minimum program that relies on `P`. Consider:

```
axiom is_null_terminated(const char* str)
    [[pre: str != nullptr]]
{
    while (*str != '\0') ++str;
}
```

This would mean that the code in braces if it were to be evaluated is required to:

1. have no UB or other contract violation,
2. finish (not run forever).

Similar examples:

```
template <input_iterator It, sentinel_for<It> S>
axiom is_reachable(It begin, S end)
{
    while(begin != end) ++begin;
}

axiom points_to_valid_memory(auto* p)
    [[pre: p != nullptr]]
{
    *p;
}

axiom is_deletable(auto* p)
    [[pre: p != nullptr]]
{
    delete p;
}
```

The code inside the body has no effect on the run-time code: it is only meant to be a feedback for static analysis. Inexpressible predicates have the same constraints as functions declared with `inline`. A program is ill-formed NDR if two TUs see different definitions of this function.

These additional features do not necessarily have to be added to the language along with inexpressible predicates. But they illustrate how inexpressible predicates have room to evolve in the future.

Name "axiom" is short and attractive, so it may not be suitable for a reserved keyword. So a different name might be required: `co_axiom`, `inexpressible`.

5. Not-yet-implemented predicates

This solution partially covers the use case where I have only provided a declaration for my predicate but don't have a precondition definition yet. If the only usage of the not-yet-implemented predicate is in preconditions and postconditions, then I can declare it temporarily as an inexpressible predicate. Later, when I provide an implementation, I can change it to a normal function.

This will not work, though, if I have to negate my predicate, due to the special semantics of inexpressible predicates.

6. Problems not addressed

This proposal does not address the following problems that might require assisting the static analyzer.

6.1. Relations between inexpressible predicates

It is possible that two libraries declare two inexpressible predicates that describe the very same condition, like `library1::is_null_terminated` and `Lib2::isNullTerminated`. A static analyzer might need a hint indicating that the two conditions are equivalent. Similarly, there might be a need to hint the static analyzer that one predicate implies another. For instance, `is_sorted` implies `is_partially_sorted`. This is similar to functionality described in [N2887]. This is a possible future extension, but we do not propose it at this time, as this information can be fed to static analyzers by other means than C++ source code, through analyzer-specific configuration files.

6.2. Side effects in predicates

Another often quoted feature is the ability to allow expressions with side effects in contract predicates and provide a guarantee that they are never evaluated. For instance a requirement that a range has size equal to 1 could be expressed as:

```
template <input_iterator It, sentinel_for<It> S, invocable<value_type<It>> F>
void for_one(It begin, S end, F f)
    [[pre: ++begin == end]];
```

Again, this is a possible future extension, however, we do not see a compelling reason to provide this. `input_iterators` have been mentioned as a use case; but we can think of no single precondition or postcondition in STL that would benefit from such capability. algorithms constrained on `input_iterators` usually just traverse the range, and the only requirement for this to work is `is_reachable`, which has to be declared inexpressible anyway.

6.3. Very, very, very, expensive predicates

Some predicates can be implemented, sometimes quite easily, but are so computationally that we would never want them to be evaluated, even in `audit` builds. For instance, comparing if two regular expressions are equivalent is a PSPACE-complete problem.

This proposal does not address this problem directly, but gives a workaround. Even if you can provide a body for your predicate, also provide the inexpressible version thereof:

```
bool equivalent(regex const& r, regex const& s) { /* implementation */; }

axiom symbollically_equivalent(regex const& r, regex const& s);
```

And use the latter definition to employ the static analyzer for correctness checking based on symbols.

This does not, however, handle the situation where a predicate is toooo expensive only in certain contexts. For instance, equality of hash-maps has quadratic complexity, which can be evaluated as predicates in some functions that are known to take small maps. But in other functions, we know we never want to evaluate them even in `audit` builds. In these cases being "too expensive" is a property of a particular contract statement rather than of a predicate.

Such use case, if we find it worth addressing, could be handled by additional build level `never`, but it still seems worthwhile to keep the two things separate: an inexpressible predicate and "never evaluate this precondition here".

7. Acknowledgments

Gor Nishanov has reviewed this proposal and offered a valuable feedback. Andrew Sutton and Joshua Berne, Tomasz Kamiński and John Lakos offered a number of improvements to this paper. Marcin Grzebieluch suggested covering the relationships between predicates.

8. References

1. B. Stroustrup, A. Sutton, et al., "A Concept Design for the STL", (N3351, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3351.pdf>).
2. Gabriel Dos Reis, Bjarne Stroustrup, Alisdair Meredith, "Axioms: Semantics Aspects of C++ Concepts", (N2887, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2887.pdf>).
3. Lisa Lippincott, "Procedural function interfaces", (P0465r0, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0465r0.pdf>).
4. G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++", (P0542r5, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/P0542r5.html>).
5. Joshua Berne, John Lakos, "Contracts That Work", (P1429r3, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1429r3.pdf>).